dependency is a `DbContext` that has been registered with the scoped lifetime, you should ensure your service is also registered with a scoped lifetime. Otherwise, you could end up suffering from an issue known as *captive dependencies*. This issue arises when dependencies are registered with a shorter lifetime than their consumer. The DI container will raise an `InvalidOperationException` if you try to consume a scoped service from within a singleton, but you receive no such protection against consuming a transient service from within a singleton. If you inject a transient service into another service that is then registered as a singleton, the dependencies also effectively become singletons because the consuming service will only ever have its constructor called once during the lifetime of the application, and it will only ever be destroyed when the application stops.

Let's have a look at this, so you understand it clearly. Add the following code as a new C# class to the Services folder.

Listing 7.11 The `SingletonService` class

```
namespace CityBreaks.Services
{
    public class SingletonService
    {
        private readonly LifetimeDemoService _dependency;
        public SingletonService(LifetimeDemoService dependency)
        {
           _dependency = dependency;
        }
        public Guid DependencyValue => _dependency.Value;
    }
}
```

The code is simple; the `SingletonService` class takes your existing `LifetimeDemoService` as a dependency and uses it to generate a value. Now you need to register the `SingletonService` as a